

ASC SURGERY CENTER SYSTEM

Complete Build Status, Pending Scope & Profitability Control Report

Prepared for: Japnaaz Software / ASC Medical System

Prepared date: 7 May 2026

Blunt verdict: The system is a broad ASC management MVP/prototype. Many screens and data structures are already built, but the product is not yet production-ready for real ASC operations because security, compliance, payer contracts, payment posting, AR/denial workflows, and true case-level profitability are incomplete.

Document Purpose

This document summarizes what has already been built in the ASC surgery center system, what still needs to be built, and the profitability criteria that must be implemented inside the system so the software can help an ASC control operations, cash flow, and margins.

The report is based on inspection of the uploaded project ZIP named ascmedical.zip and supporting industry references for ASC payment, benchmarking, OR utilization, and revenue-cycle metrics.

Executive Summary

Area	Current Status	Business Meaning
UI / Feature Breadth	60-70% built	Many operational screens exist; this is not just a blank starter app.
ASC Business Logic	35-45% built	Scheduling and estimated case economics exist, but full ASC workflow control is incomplete.
Profitability Intelligence	30-40% built	Estimated margins exist, but true collected-profit logic is not complete.
Revenue Cycle Readiness	20-30% built	Claims tracking exists, but not full billing, ERA/EOB, AR, denial, underpayment, and payment posting workflow.
Security / Compliance Readiness	15-25% built	Authentication and RLS are not safe enough for real patient data.
Production Readiness	25-35% built	Build works after dependency reinstall, but lint/security/architecture issues remain.

Do not connect real patient data yet. The current login and database security model is not acceptable for medical/PHI data.

Core Technical Foundation Found

Item	Finding
Framework	React + Vite frontend application.
Database / backend service	Supabase JavaScript client connected directly from frontend.
Package name	antigravity.
Main dependencies	React 19.2, Supabase JS, Recharts, Google Generative AI, jsPDF, html2canvas, xlsx, papaparse, pdfjs-dist, mammoth.
Main application file	src/App.jsx - approximately 1,442 lines.
Largest module	src/components/SurgeryScheduler.jsx - approximately 2,379 lines.
Supabase helper	src/lib/supabase.js - approximately 883 lines of DB operations.
Components found	72 files under src, including around 40+ UI/component files.
SQL/migration files	Multiple Supabase SQL files for patients, surgeries, surgeons, settings, claims, billing, permissions, OR schedule, audit logs, chatbot, financial snapshot, costs, and CPT data.

What Has Already Been Built

Application and Platform Layer

Capability	Status	Notes
React + Vite app structure	Built	Modern frontend app foundation is present.
Supabase connection	Built	Direct frontend-to-Supabase data access exists.
Mock/demo fallback data	Built	Useful for demos, but must be disabled in production.
Sidebar dashboard layout	Built	Role/menu driven navigation is present.
Role-based UI menu	Partial	Menu visibility is implemented, but backend enforcement is weak.
SQL database schema	Partial/started	Many tables and migrations exist, but security rules are too open.
Build output	Built after reinstall	Build succeeds after npm install; packaged node_modules was incomplete.

Functional Modules Present in Codebase

Module	File	Built Level	Assessment
Financial Dashboard	Dashboard.jsx	70%	Revenue/cost/profit overview exists, but needs true collected revenue and fixed-cost allocation.
Manager Dashboard	ManagerDashboard.jsx	70%	Good operational overview started.
Patient Management	PatientManagement.jsx	65%	Patient CRUD and insurance fields exist.
Surgery Scheduler / OR Log	SurgeryScheduler.jsx	75%	Large scheduling module with OR/case data, timing, CPT, and cost fields.

Surgery Schedule View	SurgerySchedule.jsx	60%	Schedule display exists.
OR Block Schedule	ORBlockSchedule.jsx	60%	Block schedule module exists.
OR Utilization	ORUtilization.jsx	65%	Utilization view exists, needs stronger analytics and enforcement.
Surgeon Management	SurgeonManagement.jsx	65%	Surgeon directory and login-related fields exist.
Surgeon Dashboard	SurgeonDashboard.jsx	60%	Surgeon-specific view exists.
Surgeon Scorecard	SurgeonScorecard.jsx	60%	Good base; needs true profit, denial, supply, and utilization metrics.
Staff Management	StaffManagement.jsx	50%	Staff module exists but is not a full labor planning system.
User Management	UserManagement.jsx	50%	User CRUD exists, but auth model must be rebuilt.
Roles & Permissions	RolePermissionManagement.jsx	45%	UI exists; backend/RLS enforcement is incomplete.
Claims Management	ClaimsManagement.jsx	45%	Claim tracking exists; not a complete RCM engine.
HCFA 1500 Form	HCFA1500Form.jsx	40%	Form component exists; claim generation and submission workflow needs work.
Patient Billing	PatientBilling.jsx	35%	Basic billing view; actual payment/posting workflow is missing.
CPT Manager	CPTManager.jsx	70%	CPT/category management exists.
CPT Auto-Updater	CPTAutoUpdate.jsx	55%	Import/update flow exists; source validation/versioning needed.
Supply Manager	SupplyManager.jsx	45%	Supply/cost model started; inventory/case usage missing.
Cost Analytics	CostAnalysis.jsx	55%	Cost analytics exists; needs actual case margin snapshots.
Settings	Settings.jsx	60%	Settings module exists.
Audit Logs	AuditLogs.jsx	30%	UI/migration present, but compliance-grade logging not complete.
AI Analyst	AIAnalystModal.jsx	45%	AI/profit helper exists; PHI-safe architecture missing.
Chatbot / Instructions	Chatbot.jsx / InstructionPanel.jsx	45%	Useful support layer, but PHI and prompt logging controls are missing.
Cancellation / Rescheduling	CancellationRescheduling.jsx	55%	Cancellation/rescheduling logic exists and needs integration into financial loss tracking.

Database and Migration Work Already Started

Database Area	Evidence in Project	Current Assessment
Patients	supabase-schema.sql, patient insurance migration	Core structure exists; access rules need hardening.
Surgeons	supabase-migration-surgeons.sql, split surgeon names migration	Surgeon module exists; surgeon login/password design needs replacement.
Users / Roles	users table, master permissions migration, role permissions schema	Useful structure; must link to Supabase Auth and enforce RLS.
Surgeries / OR	surgeries table, actual times, turnover time, rescheduled status, OR schedule migrations	Strong start for OR operations.
Billing	billing table, gross charge, write-off, claims-billing link migrations	Billing model started; actual payment posting missing.

Claims	insurance_claims, claim_line_items, claim_history migrations	Claim tracking started; clearinghouse/EDI/ERA missing.
CPT Codes	CPT category map, orthopedic CPT migration, average duration migration	CPT foundation exists; official fee/rate update workflow needed.
Supplies / Labor	supplies/labor cost SQL and status docs	Cost model started; true inventory and case usage missing.
Settings	settings migrations, MPPR setting, API key setting	Settings layer exists.
Audit Logs	audit logs migration	Started but must become mandatory for every sensitive action.
Chatbot Questions	chatbot questions migration	Support/FAQ capability started.

Technical Risks Found

Risk	Severity	Why It Matters	Required Fix
Plain-text password logic	Critical	The code checks password values from users/surgeons tables instead of using secure auth. This is unacceptable for medical data.	Replace with Supabase Auth or backend auth. Remove password columns from app-managed tables.
Open/broad RLS policies	Critical	Multiple SQL policies allow all operations using true/with check true. This can expose data if anon/auth access is misused.	Rewrite RLS by role and ownership.
.env included in ZIP	High	Environment secrets/config were packaged in the ZIP.	Remove .env from repo/ZIP and rotate keys if real.
Frontend directly controls sensitive logic	High	Frontend permission checks can be bypassed.	Move sensitive authorization and validation into RLS/server/backend functions.
AI feature privacy risk	Critical	Patient/surgery data may be sent to Gemini without PHI redaction or audit control.	Add backend AI proxy, redaction, permission checks, and prompt logs.
No mature audit enforcement	Critical	ASC systems need traceability for patient and financial changes.	Enforce audit logs for all create/update/delete actions.
Large monolithic files	High	SurgeryScheduler, App, Dashboard, ManagerDashboard are too large and harder to test.	Split into hooks, services, forms, tables, and route modules.
Lint errors remain	High	The saved lint result shows dozens of code quality issues, including duplicate keys and hook warnings.	Fix lint before production.
Large JS bundle	Medium/High	Built asset is around 3.1 MB before gzip, which can hurt loading performance.	Add code splitting and lazy loading.

What Still Needs to Be Built

Production Foundation

Pending Item	Priority	Reason
Supabase Auth or secure backend authentication	Critical	Current login is not production-safe.
Password removal/hashing migration	Critical	Do not store or compare plain passwords in app tables.
Role-based RLS policies	Critical	Real security must be enforced in the

		database/backend, not only UI.
Admin/manager/surgeon/patient access boundaries	Critical	Different roles must only see permitted patient, case, and financial data.
Audit trail for all sensitive actions	Critical	Needed for accountability, compliance, and debugging.
Backend/API validation layer	High	Scheduling, billing, claims, and permission logic need server-side enforcement.
Error monitoring/logging	High	Production operations need crash and failed-action visibility.
Test suite	High	No serious medical/business system should ship without regression tests.

ASC Operations Workflow

Workflow Area	Current State	Needed Build
Case lifecycle	Scheduling exists	Track scheduled, verified, authorized, pre-op, checked-in, in OR, recovery, discharged, coding complete, claim submitted, paid, closed.
Insurance verification	Missing/weak	Eligibility, benefits, deductible, copay, coinsurance, payer rules.
Prior authorization	Missing	Authorization status, submission date, approval/denial, expiration, CPT/payer rules.
OR conflict prevention	Partial	Enforce conflict checks on backend/database, not only UI.
Block utilization rules	Partial	Surgeon block retention/release rules and white-space reporting.
Staff/labor assignment	Partial	Nurse/tech/anesthesia staffing by case, duration, overtime, direct labor cost.
Supply usage	Partial	Preference card, picked items, actual used, wasted items, implants, lot/serial, variance.

Revenue Cycle Management

RCM Capability	Current State	Needed Build
Claim tracking	Started	Improve statuses, work queues, owner assignment, notes, follow-up dates.
Claim scrubber	Missing	Validate CPT/ICD/modifiers, payer rules, auth, patient info, NPI/taxonomy, place of service.
EDI 837 / electronic claims	Missing	Generate/export/submit standard claim files or clearinghouse payloads.
ERA/EOB posting	Missing	Import payer responses and auto-post payments/adjustments.
Payment posting	Missing	Record payer payment, patient payment, contractual adjustment, write-off, refunds.
Denial management	Weak	Denial reason, root cause, appeal status, preventability, payer/surgeon/CPT trends.
AR aging	Missing	0-30, 31-60, 61-90, 90+ buckets with follow-up workflow.
Underpayment detection	Missing	Compare actual payment to contracted allowed amount.

Profitability Criteria an ASC Must Track

An ASC becomes profitable only when collected revenue exceeds variable case costs plus allocated fixed overhead. Billed charges are not enough. Expected reimbursement is not enough. The system must track actual cash collection and true case-level margin.

Core profitability equation

Actual Collected Revenue - Direct Variable Case Costs - Allocated Fixed Cost = True Case Profit

True Case Profit / Actual OR Minutes = Profit per OR Minute

Total Collected Revenue / Completed Cases = Net Revenue per Case

Profitability Criterion	Formula / Measure	Why It Must Be in System
OR Utilization	Actual surgery minutes used / available OR minutes	Shows whether expensive OR capacity is being used or wasted.
Revenue per OR Minute	Net collected revenue / actual OR minutes	Shows whether a case/specialty is worth OR time.
Profit per OR Minute	Case profit / actual OR minutes	Better than revenue because it includes cost.
Net Revenue per Case	Collected revenue / completed cases	Shows revenue quality by CPT, payer, surgeon, specialty.
Contribution Margin per Case	Collected revenue - direct case costs	Shows whether the case is worth performing before fixed overhead.
True Case Profit	Contribution margin - allocated fixed cost	Shows whether the case helps the ASC overall.
Payer Profitability	Collected revenue and margin by payer/contract	Identifies payers that underpay or delay payment.
CPT Profitability	Margin and profit/minute by CPT/procedure group	Shows procedures to grow, renegotiate, or stop accepting.
Surgeon Profitability	Profit, utilization, cancellations, supplies, denials by surgeon	Shows which surgeons create profitable volume.
Block Utilization	Used block minutes / allocated block minutes	Controls surgeon block retention and white-space release.
Supply Cost per Case	Supplies + implant + medication cost per case	Stops margin leakage.
Denial Rate	Denied claims / submitted claims	Shows billing/coding/auth problems.
Net Collection Rate	Payments / expected collectible amount	Shows how well the ASC collects what it should.
Days in AR	Accounts receivable / average daily revenue	Shows cash-flow speed and billing team effectiveness.
AR Over 90/120 Days	Old AR / total AR	Shows cash stuck and likely bad debt.

Case-Level Profitability Snapshot

The system should lock a profitability snapshot when a case is completed and update financial fields as payments arrive. This prevents dashboards from changing randomly when CPT rates or supply costs are edited later.

Field Group	Fields Needed
Case identity	case_id, patient_id, surgeon_id, payer_id, facility/location, specialty, primary CPT, all CPTs, modifiers.
Timing	scheduled start/end, actual in-room, procedure start/end, procedure minutes, turnover minutes, overtime minutes.
Revenue	gross charge, expected allowed amount, payer contracted rate, Medicare rate, payer payment, patient payment, total collected.
Adjustments	contractual adjustment, write-off, bad debt, refund, underpayment amount.

Costs	supplies, implants, medications, direct labor, anesthesia, room variable cost, allocated fixed cost.
Profit outputs	contribution margin, true case profit, margin percentage, revenue per OR minute, profit per OR minute.
RCM outputs	claim id, claim status, denial reason, days to bill, days to pay, AR bucket, appeal status.

Exact Database Modules to Add for Profitability

Table / Module	Purpose
payer_contracts	Stores payer, contract name, effective dates, contract status, payment rules.
payer_contract_rates	Stores CPT/procedure-level allowed amounts by payer, year, facility, modifier, specialty.
case_profitability_snapshots	Locks actual case-level economics for reporting and audit.
payment_postings	Stores payer and patient payments by claim/case.
adjustments_writeoffs	Stores contractual adjustments, write-offs, bad debt, refunds.
denials	Tracks denial reason, preventability, appeal status, owner, financial impact.
ar_aging_snapshots	Stores daily/monthly AR aging buckets and trends.
prior_authorizations	Tracks authorization requirements, status, approval/denial, expiration.
insurance_verifications	Stores eligibility, benefits, deductible, copay, coinsurance, payer response.
case_supplies_used	Tracks supplies/implants/medications actually consumed by a case.
surgeon_block_performance	Tracks surgeon block usage, release, cancellation, overrun, revenue, profit.
fixed_costs	Tracks rent, admin salary, software, debt, utilities, insurance, billing cost.
daily_profitability_snapshots	Facility-level daily revenue, cost, profit, utilization, AR movement.
monthly_profitability_snapshots	Board/owner-level monthly P&L view.

Dashboard Requirements

Owner / CEO Dashboard

Metric	Purpose
Monthly collected revenue	Shows real money received, not billed fantasy.
Monthly net profit / EBITDA	Shows operating performance.
Case volume	Shows growth and surgeon utilization.
Average net revenue per case	Shows revenue quality.
Average profit per case	Shows real margin.
Profit per OR minute	Best metric for OR economic quality.
OR utilization	Shows facility capacity usage.
Cancellation rate	Shows lost revenue and scheduling leakage.
AR aging	Shows cash stuck in claims/patient balances.

Denial rate	Shows billing/auth/coding quality.
Net collection rate	Shows payer/payment effectiveness.
Top/bottom CPTs by profit	Tells what to grow or stop.
Top/bottom surgeons by profit	Tells where to focus relationship management.
Supply cost per case	Shows material cost leakage.

Manager Dashboard

Metric / Work Queue	Purpose
Today schedule and room status	Live operational control.
OR utilization today/this week	Capacity management.
Late starts and overruns	Operational delay control.
Turnover delays	Room efficiency control.
Cancelled/rescheduled cases	Lost revenue and workflow quality.
Missing documents / consents	Pre-op readiness.
Missing insurance verification	Prevents claim denial and day-of-surgery disruption.
Missing authorization	Prevents avoidable denials.
Cases ready for billing	Controls days to bill.
Inventory/supply alerts	Prevents procedure delay and cost leakage.

Surgeon Scorecard

Metric	Purpose
Cases per month	Volume contribution.
Block utilization	Whether allocated OR block is used.
Average case duration vs expected	Planning accuracy and OR efficiency.
Average start delay	Operational discipline.
Average turnover time after surgeon cases	Room reset impact.
Net revenue per case	Revenue quality.
Profit per case	True economic value.
Profit per OR minute	Best surgeon economics measure.
Supply/implant cost per case	Cost behavior.
Denial rate by surgeon/CPT	Documentation/coding/authorization risk.
Cancellation rate	Lost block value.
Patient satisfaction and adverse events	Quality and reputation impact.

Billing / RCM Dashboard

Metric / Queue	Purpose
Claims created/submitted/accepted/denied/paid	Pipeline visibility.
Days to bill	Billing speed.

Days in AR	Cash-flow speed.
AR 0-30 / 31-60 / 61-90 / 90+	Collections prioritization.
Denial reasons	Root cause fixing.
Appeal status	Recovery workflow.
Underpaid claims	Contract enforcement.
Patient balance collection	Patient responsibility control.
Payer performance	Negotiation and contract decisions.

Recommended Development Roadmap

Roadmap Phase	Priority	Scope
Phase 1: Security & foundation	Critical	Remove .env from repo/ZIP; rotate keys; replace custom login with Supabase Auth; remove plain-password logic; rewrite RLS; disable demo login; enforce audit logs; add backend validation.
Phase 2: Profitability Core	Critical	Add payer contracts, payer CPT rates, case profitability snapshots, payment posting, fixed cost allocation, true case profit, profit per OR minute.
Phase 3: RCM Core	High	Build insurance verification, prior auth, claim scrubber, payment posting, denial management, AR aging, underpayment detection.
Phase 4: Operations Control	High	Complete case lifecycle, OR conflict engine, block release rules, staff assignment, supplies/implants actual usage, inventory controls.
Phase 5: Reporting and Quality	Medium/High	QAPI, incident reporting, patient satisfaction, surgeon satisfaction, clinical quality metrics, benchmarking export.
Phase 6: AI safely	Medium/High	Backend AI proxy, PHI redaction, prompt/response logs, role-gated AI access, no raw PHI to third-party models without policy.

Immediate Next 20 Tasks

1. Remove .env from repository/package and rotate any real Supabase/API keys.
2. Replace custom users/surgeons password login with Supabase Auth or secure backend authentication.
3. Remove app-managed plain password columns and password forms from user/surgeon management.
4. Rewrite all USING (true) and WITH CHECK (true) RLS policies with role-specific rules.
5. Create auth_profiles table linked to auth.users for admin/manager/surgeon/patient profiles.
6. Move sensitive writes for scheduling, claims, billing, and payments into backend functions/serverless actions.
7. Add audit logging middleware/function for every create, update, delete, login, export, and AI action.
8. Fix all lint errors, including duplicate keys and React hook dependency issues.
9. Split SurgeryScheduler.jsx into smaller components and business-logic hooks.
10. Split Dashboard.jsx and ManagerDashboard.jsx into reusable analytics cards and data services.
11. Add payer_contracts and payer_contract_rates tables.
12. Add case_profitability_snapshots table and snapshot generation logic at case completion.
13. Add payment_postings and adjustments_writeoffs modules.
14. Add AR aging dashboard and daily AR snapshot job.
15. Add denial management with denial reason, preventability, appeal status, and financial impact.
16. Add insurance verification and prior authorization modules before surgery day.

17. Add OR white-space and surgeon block utilization reports.
18. Add actual supplies/implants used per case and variance from preference card.
19. Add code splitting/lazy loading to reduce initial JS bundle size.
20. Add regression tests for login, permissions, patient privacy, scheduling conflicts, claim lifecycle, and case profitability.

ASC Profitability Logic to Implement in System

Case profitability calculation

Gross Charge - Contractual Adjustment = Expected Allowed Amount

Expected Allowed Amount - Denial Risk Adjustment - Bad Debt Estimate = Expected Net Revenue

Actual Payer Payment + Actual Patient Payment = Actual Collected Revenue

Actual Collected Revenue - Supplies - Implants - Medications - Labor - Anesthesia - Room Variable Cost = Contribution Margin

Contribution Margin - Allocated Fixed Cost = True Case Profit

OR economics calculation

OR Utilization % = Actual Surgery Minutes Used / Available OR Minutes

Revenue per OR Minute = Net Collected Revenue / Actual OR Minutes Used

Profit per OR Minute = True Case Profit / Actual OR Minutes Used

Lost OR Revenue Opportunity = Unused OR Minutes x Average Revenue per OR Minute

Facility-level monthly calculation

Monthly ASC Profit = Total Collected Revenue - Direct Variable Costs - Payroll - Rent/Lease/Mortgage - Equipment Lease - Insurance - Software - Utilities - Billing Cost - Debt Payments - Other Overhead

Break-Even Criteria to Add

Question the System Must Answer	Required Calculation
How many cases per month are needed to break even?	Monthly fixed costs / average contribution margin per case.
How many OR minutes must be sold every month?	Monthly fixed costs / average contribution margin per OR minute.
Which payer contracts are loss-making?	Payer-level collected revenue minus variable costs and allocated overhead.
Which CPTs should the ASC grow?	CPT-level profit per case and profit per OR minute.
Which CPTs should be renegotiated or avoided?	CPTs with low/negative margin or high denial/AR delay.
Which surgeons deserve more block time?	Surgeons with high utilization, high profit/minute, low cancellations, and low denial rate.
Which surgeons should lose block time?	Low utilization, frequent cancellations, high supply waste, low profit/minute.
Where is cash stuck?	AR aging by payer, CPT, surgeon, claim owner, and denial reason.

Suggested Screen: ASC Profitability Control Center

This should become the main strategic module. It should combine OR utilization, payer contracts, case profitability, AR, denials, payment posting, surgeon performance, CPT performance, and fixed-cost break-even into one control center.

Section	Widgets / Outputs
Top summary	Collected revenue, net profit, EBITDA, case volume, OR utilization, profit per OR minute, AR over 90 days, denial rate.
Case margin table	Case, surgeon, CPT, payer, actual minutes, collected revenue, total cost, profit, profit/minute, AR status.
CPT profitability	Top 10 profitable CPTs, bottom 10 CPTs, loss-making CPTs, high-revenue/low-profit CPTs.
Surgeon profitability	Block usage, case count, revenue, profit, cancellations, supply variance, denial rate.
Payer profitability	Expected vs actual payment, denial rate, days to pay, net collection rate, underpayment.
OR utilization	Available minutes, used minutes, white space, released blocks, overtime, turnover.
Revenue cycle	AR aging, claims stuck, denials, appeals, underpayments, patient balances.
Break-even engine	Monthly fixed cost, contribution margin, required cases, current gap, profit forecast.

Clean Product Definition

Recommended positioning: This should not be sold as only a scheduler. It should be positioned as an ASC Profitability Operating System: schedule cases, control OR time, track costs, submit claims, collect payments, and show which cases/surgeons/payers actually make money.

Current Product	Better Product Direction
ASC management dashboard	ASC profitability and operations control system.
Tracks surgeries and estimated revenue	Tracks full revenue lifecycle from authorization to collected payment.
Shows dashboards	Shows decisions: grow, stop, renegotiate, release block, chase claim, reduce cost.
Frontend role permissions	Database/backend-enforced role and PHI security.
Estimated case economics	Locked actual case profitability and AR/denial impact.
Chatbot/AI helper	PHI-safe AI analyst with redaction, audit logs, and permission controls.

Final Verdict

You have built a serious MVP foundation with many ASC modules already present. The work is not wasted. The product has enough breadth to become valuable.

However, the system is not yet a true ASC profitability/control platform. The current product mainly covers patient, surgeon, surgery, CPT, scheduling, dashboards, claims tracking, and estimated financials. It does not yet fully control the real money cycle: payer contract, authorization, coding, claim, payment, denial/write-off, AR, and actual case margin.

Correct next move: stop adding random screens and build the Profitability + RCM Core. That is the difference between a nice-looking ASC app and software that can actually help a surgery center become profitable.

Evidence and Industry References

These references were used to validate the ASC profitability and benchmarking criteria included in this report. Links are listed as plain URLs for easy copying.

Source	Why Used	URL
CMS - Ambulatory Surgical Center (ASC) Payment	CMS describes ASC approved HCPCS codes, payment rates, and ASC payment groups used for Medicare facility payment.	https://www.cms.gov/medicare/payment/prospective-payment-systems/ambulatory-surgical-center-asc
CMS - Ambulatory Surgical Center Payment: January 2026 Update	CMS January 2026 update includes ASC payment system code/device updates effective January 1, 2026.	https://www.cms.gov/files/document/mm14359-ambulatory-surgical-center-payment-january-2026-update.pdf
ASCA Benchmarking	ASCA benchmarking supports ASC comparison across quality, operational, staffing, and financial performance areas.	https://www.ascassociation.org/clinical-ops-benchmarking
HST Pathways - Three Metrics to Drive ASC Growth Strategy	Defines and emphasizes OR utilization, revenue per OR minute, and net revenue per case as ASC growth metrics.	https://www.hstpathways.com/blog/three-metrics-to-drive-your-ascs-growth-strategy/
HST Pathways - Improving OR Utilization	Emphasizes OR utilization, revenue per OR minute, and net revenue per case for ASC efficiency and growth.	https://www.hstpathways.com/content-bundles/improving-or-utilization/
SIS - ASC Benchmarking: Doing It Better and Making the Work Count	Discusses clinical, operational, financial, patient-experience metrics and benchmarking categories for ASCs.	https://blog.sisfirst.com/asc-benchmarking-doing-it-better-and-making-the-work-count
Tebra - Revenue Cycle KPIs	Discusses revenue-cycle KPIs including days in receivables outstanding and net collection rate.	https://www.tebra.com/theintake/practice-operations/rcm-and-claims/key-performance-indicators-for-the-revenue-cycle
Serbin Medical Billing - 20 ASC Revenue Cycle KPIs	Lists ASC-specific revenue-cycle KPIs including insurance verification/authorization and patient financial counseling.	https://www.serbinmedicalbilling.com/the-20-asc-revenue-cycle-kpis-you-should-be-tracking

Appendix A: Codebase Evidence Snapshot

Evidence Item	Finding
Project ZIP	ascmedical.zip
Framework	React + Vite
Database integration	Supabase JS client
Main app file	src/App.jsx - approx. 1,442 lines
Largest component	src/components/SurgeryScheduler.jsx - approx. 2,379 lines
Dashboard components	Dashboard.jsx approx. 1,002 lines; ManagerDashboard.jsx approx. 1,191 lines
DB service file	src/lib/supabase.js approx. 883 lines
RLS issue count found	At least 19 instances of USING (true) / WITH CHECK (true) in SQL files.
Password logic found	Login methods compare email/password directly against users and surgeons tables.
Lint status found	Saved lint result showed 48 problems: 43 errors and 5 warnings. Earlier inspection showed similar high lint debt.
Build bundle	Built JS asset around 3.1 MB before gzip; code splitting recommended.

Appendix B: Module Cleanup List

The following files may be duplicate, old, or in need of confirmation before production cleanup. Do not delete blindly; first verify imports and intended usage.

Potential Cleanup Item	Reason to Review
CPTManagement.jsx vs CPTManager.jsx	Possible duplicate/old CPT module naming.
CPTDurationUpdater.jsx vs CPTAutoUpdate.jsx	Possible overlapping CPT duration/update features.
SurgeonManager.jsx vs SurgeonManagement.jsx	Possible duplicate surgeon management module.
PatientRegistration.jsx vs PatientManagement.jsx	May be old or separate patient flow.
CategoryManagement.jsx	Check whether still used after CPT category mapping.
Hero/Header components	May be demo/landing components rather than internal ASC app screens.